

QECTOR Decoder Workbench

USER MANUAL

macOS Edition

Application version 3.5.1

Decoder backend qector-decoder-v3 0.6.9 (bundled; minimum supported 0.6.2)

47-tool MCP server, 13 decoders, 9 code families

Guillaume Lessard / iD01t Productions. <https://www.qector.store>

Generated 2026-07-26

Contents

1. Introduction	3
2. System requirements	3
3. Installation	3
Install from the disk image	3
4. First launch	3
5. The graphical interface	4
6. Working with codes	5
7. Decoders	5
8. Decoding, benchmarking and streaming	6
Decoder Lab	6
Benchmark	6
Batch and Streaming	6
9. Hardware, diagnostics and export	6
Hardware	6
Diagnostics	6
Documentation export	6
10. The MCP server	8
11. Environment variables	8
12. Troubleshooting	8
13. Licensing and support	8
14. Glossary	8
Appendix A. Full MCP tool list	10

1. Introduction

QECTOR Decoder Workbench is a professional desktop suite for building quantum error-correcting codes and analysing their decoders. It combines a graphical workbench of seven feature tabs and a live console, a resilient decoding backend, a multi-format documentation generator, and a headless server that exposes 47 tools over the Model Context Protocol (MCP) for use by automation and language-model agents.

The suite is powered by qector-decoder-v3, a compiled Rust decoding engine with a Python API. It offers 13 single-shot decoders across 9 code families, batch and streaming decode, hardware detection, and fully reproducible seeded runs.

Note. This edition requires an internet connection on first launch to download and install the decoder backend from PyPI. After that it works offline. Reported logical error rate, throughput and latency are simulation outputs that depend on the seed, the hardware and the workload; regenerate them for your own setting before quoting.

2. System requirements

- macOS 11 (Big Sur) or newer, Apple Silicon or Intel.
- About 300 MB of free disk space.
- Python 3.10+ and pip are required for the decoder auto-install.
- An internet connection is required on first launch so the application can download and install qector-decoder-v3 from PyPI.

3. Installation

Install from the disk image

- Open QectorWorkbench.dmg and drag QECTOR Workbench to Applications.
- First launch: right-click the app and choose Open, then confirm, so Gatekeeper allows the unsigned build to run.

4. First launch

The application starts immediately. The decoder backend is downloaded and installed automatically from PyPI on first launch, then loaded at startup. The window title and the status bar show the active decoder version.

Start the graphical application with QECTOR Workbench.app. To run the headless MCP server, which needs no display, use:

```
/Applications/QectorWorkbench.app/Contents/MacOS/QectorWorkbench --mcp
```

Note. To move to a newer decoder, install a newer release of the application or run the updater. The decoder itself is fetched from PyPI at first launch.

5. The graphical interface

The workbench presents the following tabs. A typical session builds a code in the Code Explorer, then decodes, benchmarks or streams it in the other tabs.

Code Explorer	Pick one of the nine code families, set the distance, and inspect the code: qubit and check counts, code distance, and a readable Tanner graph. This is where you build the code that the other tabs operate on.
Decoder Lab	Run a single seeded decode with any of the ten decoders. The panel reports the correction weight, whether the correction reproduces the observed syndrome (syndrome_valid), and whether a logical operator was flipped. If a chosen decoder cannot handle the selected code, a resilient fallback recovers automatically and records the full attempt trace.
Benchmark	Sweep a decoder over many seeded shots and read throughput (decodes per second), latency percentiles, and the logical error rate for that workload.
Batch and Streaming	Decode a batch of syndromes on the CPU, CUDA or OpenCL backend, and run a sliding-window streaming session with per-round commit telemetry.
Hardware	Detect CUDA and OpenCL availability, show system information, and read a decoder recommendation that never suggests a decoder the selected code cannot use.
Diagnostics	Run a full self-test of the environment, probe which decoders work for a given code, and exercise the resilient decode path with a complete trace.
Documentation	Export a provenance-stamped record of the current code in six formats: Markdown, JSON, HTML, LaTeX, PDF and SVG.
Console	A live, severity-coloured log of every operation, kept in one place for review and troubleshooting.

6. Working with codes

Open the Code Explorer, choose a family, set the distance (minimum 3), and build. The 9 families are:

repetition	Repetition Code	graphlike	1D chain that protects against bit-flip errors. The simplest teaching code.
ring	Ring Code	graphlike	Periodic 1D code (repetition on a ring). Good for streaming and cyclic examples.
rotated_surface	Rotated Surface Code	graphlike	Compact rotated surface code; the standard workhorse for 2D QEC studies.
unrotated_surface	Unrotated Surface Code	graphlike	Classic surface code layout with boundary stabilisers.
toric	Toric Code	graphlike	Surface code on a torus (periodic boundaries); two logical qubits.
heavy_hex	Heavy Hex Code	graphlike	IBM heavy-hexagon layout used by superconducting hardware.
bicycle	Bicycle Code (qLDPC)	qLDPC (non graphlike)	A qLDPC code that every applicable decoder can handle.
bivariate_bicycle	Bivariate Bicycle Code (qLDPC)	qLDPC (non graphlike)	The IBM bivariate-bicycle family, e.g. the $[[72,12,6]]$ gross code.
hypergraph_product	Hypergraph-Product Code (CSS)	graphlike	Constructs a CSS code from two classical codes; graphlike here.

Note. The two qLDPC families (bicycle, bivariate_bicycle) are not graphlike. Decode them with bp_osd, sparse_blossom, blossom, hybrid, predecoded, auto or auto_router. The resilient decode path selects a working decoder automatically if you pick one that does not apply.

7. Decoders

These single-shot decoders are available by exact name in the Decoder Lab, Benchmark and Diagnostics tabs and in the MCP server.

union_find	Fast graphlike matching for surface, repetition and ring codes. Not for qLDPC.
fast_union_find	SIMD-accelerated union-find; highest throughput on graphlike codes. Not for qLDPC.
blossom	Exact minimum-weight perfect matching. Best accuracy on graphlike codes; slower.
sparse_blossom	Region-growing matcher; strong accuracy with better scaling than exact blossom.
bp_osd	Belief propagation with ordered-statistics decoding. The go-to decoder for qLDPC codes.
auto	Policy decoder: inspects the code and dispatches a suitable concrete decoder automatically.
hybrid	Neural pre-decoder feeding sparse blossom; adaptive edge weights for enriched decoding.
lookup_table	Exact table for small codes only (refused above 20 checks). Constant-time lookups.

predecoded	A pre-decoding stage in front of a matcher to reduce work on likely error patterns.
auto_router	Routes graphlike codes to matching and qLDPC codes to BP-OSD; safe default everywhere.
hybrid_cascade	Hybrid Cascade , Union-Find pre-filter + Blossom/BP-OSD escalation (v0.6.9): trivial syndromes resolve at UF speed, hard ones escalate to the accurate decoder. Exposes prefilter_hits / escalations / prefilter_hit_rate stats. Graphlike codes only (UF pre-filter).
gnn_belief_matching	GNN Belief Matching , GNN-predicted per-qubit weights guide a weighted matching decode (v0.6.9 GNNBeliefMatcher); a built-in faithfulness check falls back to plain MWPM so corrections stay syndrome-valid. Requires qector_decoder_v3 >= 0.6.9; graphlike codes.
belief_matching	Belief Matching , sum-product BP posteriors reweight an exact Blossom matching step (Higgott et al. 2023), recovering error-correlation information plain MWPM discards. Faithfulness-checked; falls back to plain MWPM so corrections stay syndrome-valid.

8. Decoding, benchmarking and streaming

Decoder Lab

Select a decoder, set a physical error rate and a seed, and run a single decode. The result reports the correction weight, syndrome_valid (whether the correction reproduces the syndrome), and logical_failure (whether a logical operator flipped). A seed makes the run exactly reproducible.

Benchmark

Sweep the chosen decoder over many seeded shots to read throughput (decodes per second), latency percentiles, and the logical error rate for that workload.

Batch and Streaming

Decode a batch of syndromes on the CPU, CUDA or OpenCL backend, or run a sliding-window streaming session with per-round commit telemetry. The CPU backend is the default and works everywhere; GPU backends are used only when present and healthy.

9. Hardware, diagnostics and export

Hardware

Detects CUDA and OpenCL availability and system information, and gives a decoder recommendation that never proposes a decoder the selected code cannot use.

Diagnostics

Runs a full environment self-test, probes which decoders work for a given code, and exercises the resilient decode path with a complete attempt trace. Use it first when anything looks wrong.

Documentation export

The Documentation tab writes a provenance-stamped record of the current code in six formats (Markdown, JSON, HTML, LaTeX, PDF, SVG) into your data directory. For a very large code the SVG export falls back

to a minimal valid file rather than failing.

10. The MCP server

The application ships a server that speaks newline-delimited JSON-RPC 2.0 over standard input and output, protocol version 2024-11-05. Start it with the `--mcp` flag. A client sends an initialize request, then an initialized notification, then calls `tools/list` to enumerate the 47 tools or `tools/call` to invoke one.

```
/Applications/QectorWorkbench.app/Contents/MacOS/QectorWorkbench --mcp
```

A complete developer walkthrough is in the QECTOR MCP Integration Guide, and a machine-readable description of every tool is in `QECTOR_LLM_Manual.json`.

11. Environment variables

QECTOR_DATA_DIR	Relocate all per-user data (logs, exports, settings) to a chosen directory.
QECTOR_DISABLE_OPENCL	Set to 1 to skip OpenCL probing on systems with unstable GPU drivers.
QECTOR_SILENT	Set to 1 to suppress the backend startup notice.

The per-user data directory for this edition is `~/Library/Application Support/QectorWorkbench`.

12. Troubleshooting

The app does not appear after launch.	Wait a few seconds on first run while the bundle initialises. Check the Console tab, or run from a terminal to see messages.
A decoder is refused for a qLDPC code.	Choose <code>bp_osd</code> , <code>blossom</code> , <code>sparse_blossom</code> , <code>hybrid</code> , <code>predecoded</code> , <code>auto</code> or <code>auto_router</code> , or let the resilient path pick a working decoder.
The GPU (CUDA or OpenCL) backend reports unavailable.	This is expected without a supported GPU and healthy driver. The CPU backend decodes fully. Set <code>QECTOR_DISABLE_OPENCL=1</code> if a driver misbehaves.
Benchmarks differ between machines.	Throughput, latency and logical error rate are simulation outputs; they depend on hardware, seed and workload. Regenerate them for your setting.

13. Licensing and support

Academic and personal use is free. Institutional and commercial licences are available; see <https://www.qector.store>. The bundled `qector-decoder-v3` backend is separately licensed. Consult the EULA shipped with the application for full terms.

Project home and support: <https://www.qector.store>. Attribution: Guillaume Lessard / iD01t Productions.

14. Glossary

Stabiliser code	A quantum error-correcting code defined by a set of commuting parity checks (stabilisers) whose measurement reveals errors without disturbing the encoded information.
Syndrome	The pattern of parity-check outcomes. A decoder maps a syndrome to a correction that explains it.
Decoder	An algorithm that turns a syndrome into a correction. QECTOR ships ten.
Code distance	The size of the smallest error that can cause an undetected logical failure. Larger distance means stronger protection.
Graphlike code	A code whose errors can be represented as edges in a graph, so matching decoders (union-find, blossom) apply directly.
qLDPC code	A quantum low-density parity-check code (here bicycle and bivariate bicycle). Not graphlike; decode with BP-OSD or the routed decoders.
Logical error rate	The simulated probability that a decode leaves an undetected logical error. A simulation output, not a fixed benchmark.
MCP	Model Context Protocol. A JSON-RPC interface that lets automation and language-model agents drive the workbench headlessly.

Appendix A. Full MCP tool list

All 47 tools registered by the server, alphabetical.

analyze_code_family	Analyze a code family with an example code instance
batch_decode	Batch-decode sampled syndromes on cpu/cuda/opencv via backend.run_batch_decode
batch_decode_gpu	Batch-decode on an explicit compute backend (cpu/cuda/opencv) with honest availability reporting , unavailable GPU backends return status='unavailable' with a reason, never fake results
belief_match_decode	Convenience seeded decode pinned to the 0.6.9 belief_matching kind (BP posteriors + exact MWPM with faithfulness fallback)
benchmark_decoder	Benchmark a decoder on a code family via backend.run_benchmark (latency percentiles, throughput, logical error rate)
check_updates	Check PyPI for newer app / decoder-backend releases and return install-aware upgrade instructions
clear_results	Clear all stored benchmark results
compare_benchmarks	Compare stored benchmark results side by side (throughput, p99 latency, logical error rate)
compat_report	Report ecosystem-integration availability (stim/sinter/pymatching/qiskit/ldpc) and research components
compatible_decoders	Live probe: which decoder kinds construct and produce a syndrome-verified correction on this code
decode_single	Run one seeded decode and report correction weight, syndrome validity and logical failure
decode_syndrome	Decode an explicit 0/1 syndrome (length n_checks) with a chosen decoder; syndrome_valid is the GF(2) re-check, logical_failure is null (no reference error exists, so it is unknowable)
decode_with_options	Seeded decode with validated per-decoder construction options (bp_osd bp_method/osd_order, hybrid_cascade escalation, GNN architecture); reports options_applied honestly
delete_resource	Delete a resource by ID
diagnostic_decode	Rich single decode via the backend's native decode_with_diagnostics (matched weight, backend used, internal fallback, timing, logicals)
export_benchmark	Export a stored benchmark result (by result_id) to the export directory
generate_documentation	Generate code documentation files
get_code_properties	Get properties of a code family
get_config	Get current server configuration
get_decoder_info	Get information about a decoder
get_hardware_info	Get hardware/backend availability
get_resource	Get a specific resource by ID
get_resources	List all resources
get_results	Get stored benchmark results (most recent first-in order)
get_statistics	Get server statistics

get_system_info	Get system information
gnn_belief_match_decode	Convenience seeded decode pinned to the 0.6.9 gnn_belief_matching kind with optional GNN architecture overrides (gnn_hidden_size, gnn_n_layers)
hybrid_cascade_stats	Batch-decode through the hybrid_cascade decoder and expose its live cascade statistics (prefilter_hits, escalations, hit rate, throughput, syndrome-match rate, logical error rate)
list_clients	List registered clients
list_code_families	List available code families
list_codes	List workbench code families plus the backend's native code catalogue
list_decoders	List available decoders
list_tools	List all available MCP tools
mcp_status	Get MCP server status
native_recommend	Backend-native decoder recommendation (qector_decoder_v3.recommend) with the mapped workbench decoder_kind
native_streaming	Native hardware-accelerated sliding-window streaming decode (qector_decoder_v3.sliding_window_decode) with per-round validity + telemetry
neural_predecoder_train	Train the NeuralPredecoder research/lab MLP on seeded (syndrome, error) pairs and evaluate on a disjoint held-out stream (exact-match, bit accuracy, syndrome validity, LER)
probe_decoders	Probe which decoders produce a valid (syndrome-verified) correction for a code , a self-test across every wired decoder
recommend_decoder	Recommend a decoder for a code/priority using detected hardware
register_client	Register a client
reset_config	Reset configuration to defaults
resilient_decode	Single decode with automatic multi-decoder fallback and a full attempt trace (autodebug.resilient_single_decode)
run_benchmark	Run a benchmark and store the result under a generated result_id
self_diagnostics	Run a full environment/decoder/hardware self-diagnostics report (autodebug.run_self_diagnostics)
set_config	Merge key/value pairs into the server configuration
stream_decode	Run a sliding-window streaming decode session via backend.run_streaming_session
version_info	Live app + decoder-backend version report resolved from PyPI at call time (installed/local vs. latest, cached)